

# 报告一：Android Studio Profiler 原生性能测试报告

## 一、测试环境

| 配置项  | 详细信息                                        |
|------|---------------------------------------------|
| 测试设备 | Pixel 7 模拟器 API 34                          |
| 被测应用 | ComposeApplication 记账App                    |
| 测试工具 | Android Studio Profiler (CPU/Memory/GPU 模块) |
| 测试场景 | 冷启动、列表滑动UI渲染、长时间页面操作内存监控                    |

## 二、分场景测试内容与结果

### 测试1：应用冷启动性能测试（启动耗时指标）

#### 1.1 测试方案

- 完全杀死App后台进程，清空应用缓存，执行冷启动
- Profiler捕获 Application初始化、MainActivity加载、首页Compose布局渲染 三个阶段时间戳
- 重复冷启动10次，取平均启动耗时

#### 1.2 测试指标定义

| 指标 | 含义                          |
|----|-----------------------------|
| T0 | 点击应用图标触发进程创建                |
| T1 | Application 类 onCreate 执行完成 |
| T2 | 首页视图完全渲染、页面可交互              |

#### 1.3 测试结果

| 指标                    | 数值    |
|-----------------------|-------|
| 进程初始化耗时               | 128ms |
| Compose页面完整可交互平均启动总耗时 | 626ms |

| 指标          | 数值      |
|-------------|---------|
| 启动阶段峰值内存    | 110.5MB |
| 资源加载完成后基线内存 | 110MB   |

1.4 分析结论

启动耗时处于合格区间（标准阈值<1000ms），无阻塞主线程的同步IO操作；仅少量本地数据库同步逻辑轻微拉长初始化时间，可优化为异步预加载。



测试2：UI渲染、GPU过度绘制与帧率FPS测试

2.1 测试方案

- 进入账单列表页面，持续上下滑动列表3分钟
- 开启Profiler GPU调试工具：
  - 实时采集帧率FPS
  - 捕获界面图层绘制层数，统计过度绘制区域
  - 监控主线程Choreographer渲染耗时

2.2 测试指标

| 指标   | 标准                         |
|------|----------------------------|
| 稳定帧率 | ≥55FPS为流畅，<40FPS为卡顿        |
| 过度绘制 | 界面色块≤2层，红色区域（4层及以上）为严重过度绘制 |

2.3 测试结果

| 指标            | 数值       |
|---------------|----------|
| 常规滑动平均帧率      | 58FPS    |
| 最低瞬时帧率        | 52FPS    |
| 账单Item过度绘制层数  | 3层（文本区域） |
| Compose单次重组耗时 | <8ms     |

2.4 优化建议

移除Item嵌套多余 Box 背景容器，使用 Modifier.clip 合并图层绘制，降低GPU绘制负载，稳定帧率至60满帧。

## 测试3：内存性能测试（内存占用 + 内存泄漏检测）

### 3.1 测试方案

- 基线采集：App冷启动稳定后记录基准内存
- 加压操作：循环滑动列表、进入/退出新增账单页面，持续5分钟
- 执行多次手动GC，对比操作前后内存波谷值
- 使用Profiler内存快照抓取Activity、Compose页面、图片资源引用链，检测泄漏

### 3.2 测试结果

| 指标       | 数值                   |
|----------|----------------------|
| 稳定运行基准内存 | 110~113.5MB          |
| 内存波动特征   | 操作过程小幅震荡，GC后可回落至初始基线 |
| 内存泄漏检测   | 无页面、图片、监听回调长期引用滞留    |

### 3.3 结论

短时间手动交互场景下内存管理表现良好，对象回收机制正常，**无显性内存泄漏风险**。

## 三、Profiler测试综合总结

| 维度    | 评价                                         |
|-------|--------------------------------------------|
| 适用场景  | 短时精细化性能诊断                                  |
| 可定位问题 | 启动耗时、UI渲染卡顿、瞬时内存波动                         |
| 局限性   | 人工操作时长有限，无法模拟长时间高频循环操作，难以暴露渐进式、缓慢累积的内存泄漏缺陷 |

**结论：**Android Studio Profiler适合**短时精细化性能诊断**，可精准定位启动耗时、UI渲染卡顿、瞬时内存波动问题。